Universidad de León

LICENCIATURA EN COMPUTACIÓN

“Sistema de control de productos: implementación de control de procesos (proyecto final)”

Trabajo de la materia de:

Control de Procesos

Presenta:

David Adame Vázquez

Grupo:

801-9

Catedrático:

Ing. SILVIA MERCEDES JUAREZ CONSTANTINO

León, Gto., abril de 2026

Contenido

Descripción 2
Recursos compartidos 2
Interfaz Grafica 2
Modelo cliente consumidor: 2
Locks y sincronización 3
Medición de rendimiento 3
Cola Queue 3
Funcionamiento 4
Componentes del Sistema: 4
Base de datos: 4
Modelo de datos 4
Ejecución: 5
Prompts: 13
Prompt 1 13
Prompt 2 13
Prompt 3 13
Prompt 4 13
Prompt 5 13
Prompt 6 14
Prompt 7 14
Prompt 8 14
Prompt 9 14
Prompt 10 14
Prompt 11 15
Prompt 12 15
Prompt 13 15
Prompt 14 15
Prompt 15 16
Prompt 16 16
Prompt 17 16
Prompt 18 16
Prompt 19 16
Prompt 20 16
Código 16

Descripción

Este proyecto presenta una simulación de un sistema concurrente del mundo real, modelando el flujo de productos desde su abastecimiento hasta su entrega a los clientes.

El sistema hace uso de hilos (threads) para ejecutar simultáneamente:

- procesos de compra,
- procesos de entrega,
- procesos de reabastecimiento.

A continuación se habla a detalle de las características del sistema:

Recursos compartidos

Estos son esenciales para lograr el funcionamiento realista del sistema, los hilos acceden simultáneamente a ambos recursos:

1. **La Cola de Entregas (pending_deliveries):** Es la estructura en memoria compartida donde los hilos productores depositan información y de donde los hilos consumidores extraen los próximos pedidos que deben ser procesados de acuerdo al orden en que van llegando, se puede decir que es una estructura donde se almacenan momentáneamente los pedidos, estos esperan aquí y después de ser leídos son eliminados de aquí.
2. **La Base de Datos / Sistema de Logs (global_logger):** Todos los hilos intentan enviar mensajes de registro simultáneamente para actualizar la interfaz gráfica o guardar los cambios de estado de los repartidores y productos.

Interfaz grafica (framework Flet)

La interfaz gráfica se desarrolló en Flet, un framework creado por Google y basado en flutter utilizando las librerías y estilos especificados en la biblioteca material de Google, todo con el fin de hacerlo versátil, a pesar de ser un framework que sigue en constante desarrollo avance rápidamente e implementa cada vez más funcionalidades que lo hacen adaptable a la demanda del mundo real, cuenta con diseño moderno y actualizado.

A través de este entorno gráfico, el usuario puede iniciar, pausar y monitorear la simulación mediante un panel de logs en vivo (log_view), observando cómo interactúan los productores y consumidores.

Modelo productor consumidor:

- **Productor (comprar y restock):** Actúan como hilos productores. En la función comprar(), se generan pedidos y entregas aleatorias para los compradores. Una vez que la entrega se valida y se registra en la base de datos, el productor "coloca" (mediante .put()) este objeto de entrega en la cola de trabajo.

- **Consumidor (entregar):** Actúa como el hilo consumidor. Este hilo extrae (mediante .get()) los pedidos que están esperando en la cola. Una vez que obtiene un pedido, simula el tiempo de procesamiento, busca un repartidor disponible y le asigna la entrega para completarla. Esto permite que la recepción de compras y la asignación de repartidores trabajen a velocidades distintas sin bloquearse mutuamente.

Locks y sincronización (uso de hilos concurrentes)

El sistema no se ejecuta de forma secuencial. Se invocan hilos distintos para realizar diversas tareas independientes que interactúan con recursos del sistema simultáneamente, al invocarse, estos se ejecutan en paralelo simulando un entorno real, gestionados de forma concurrente por el sistema operativo.

Se hace uso de Locks en partes estratégicas dentro del sistema, en primer lugar se utilizan locks para bloquear el acceso de otros hilos a recursos compartidos como la base de datos, de igual manera se usó de locks para actualizar adecuadamente la interfaz

En las funciones comprar, entregar y restock, las operaciones de escritura a la base de datos se envolvieron dentro de un bloque with lock. Esto asegura que si el Hilo A está escribiendo una nueva entrega, el Hilo B debe esperar a que el Hilo A termine y libere el Lock antes de poder interactuar con la base de datos.

Medición de rendimiento

Para la medición de rendimiento del sistema, se utilizó la función time.perf_counter() de python, la cual ofrece un reloj de alta resolución ideal para medir intervalos de tiempo de ejecución. Se tomó una marca de tiempo al iniciar el ciclo de hilos (inicio) y otra al finalizar (fin). La diferencia entre ambas nos da el tiempo exacto que le tomó al programa despachar todas las tareas concurrentes, lo cual se registra en los logs del sistema.

Al inicio de la función start() de la simulación se captura la marca de tiempo inicio. Una vez que el ciclo de la simulación termina o se interrumpe por el usuario se captura el tiempo final. Finalmente los logs muestran el tiempo calculado, evaluando el tiempo que le toma al sistema para procesar el lote completo de hilos y operaciones.

Cola queue

SE hace uso de una cola como un recurso compartido entre hilos algunos hilos se encargan de almacenar en la cola referencias a la base de datos (utilizando la librería nativa de Python: pending_deliveries = queue.Queue()), mientras que otros acceden a estos de manera ordenada, todo gracias a que la cola es thread safe por lo cual no es necesario el uso de locks, internamente ya maneja sus propios mecanismos de bloqueo, asegurando que si dos hilos intentan hacer .put() o .get() exactamente en el mismo milisegundo, la cola no se corrompa ni se pierdan datos, esto es especialmente útil para protegerla de problemas como la condición carrera, abstrayendo gran parte de la complejidad de la simulación . En el caso específico del sistema la cual simula el flujo de pedidos, gracias al paradigma de FIFO (first in first out) permitiendo replicar

exactamente el funcionamiento del flujo en el cual se atienden las ordenes o los pedidos de un sistema real.

Funcionamiento

El sistema cuenta con algunas características, en primer lugar están las vistas de productos, repartidores, proveedores, clientes, entradas y ventas que muestran el contenido de la base de datos, junto con operaciones de CRUD integradas en la interfaz. Por otro lado se encuentra el funcionamiento del sistema a través de la simulación

Que activa un administrador que genera hilos productores y consumidores. Los hilos se comunican de forma segura a través de una cola y siempre que necesitan modificar el inventario o registrar una venta, solicitan acceso exclusivo mediante un Lock para guardar registros en SQLite. Todo el proceso ocurre en tiempo real y, gracias a la concurrencia, el programa funciona sin pausas, bloqueos de interfaz ni caída por saturación de recursos, arrojando al final los datos de medición de rendimiento correspondientes.

Componentes del Sistema:

Gestión de Inventario: Control de stock mediante un sistema de entradas (abastecimiento) y salidas (entregas).

Módulo de Actores: Registro y control de Productores, Compradores y Repartidores.

Sistema de Entregas: Gestión de pedidos complejos donde una entrega puede contener múltiples productos (relación maestro-detalle).

Logística de Reparto: Control de disponibilidad de repartidores en tiempo real.

Base de datos:

Como se indica en los requerimientos del ejercicio para garantizar la persistencia de la información y el historial de eventos, se implementó una base de datos relacional SQLite (almacenada localmente en el archivo requests.db). Con el fin de mejorar el diseño y hacerlo más escalable el código utiliza un ORM (Object-Relational Mapping) con SQLAlchemy para definir la estructura a través del archivo models.py, de igual manera se manejan migraciones para realizar cambios a la base de datos con alembic. Cada transacción crítica de la simulación, como la creación de una nueva entrada (crear_entrada), una nueva compra (crear_entrega), o la actualización de un pedido completado, queda registrada permanentemente en la base de datos. A continuación se listan los modelos creados de las entidades que interactúan con el sistema:

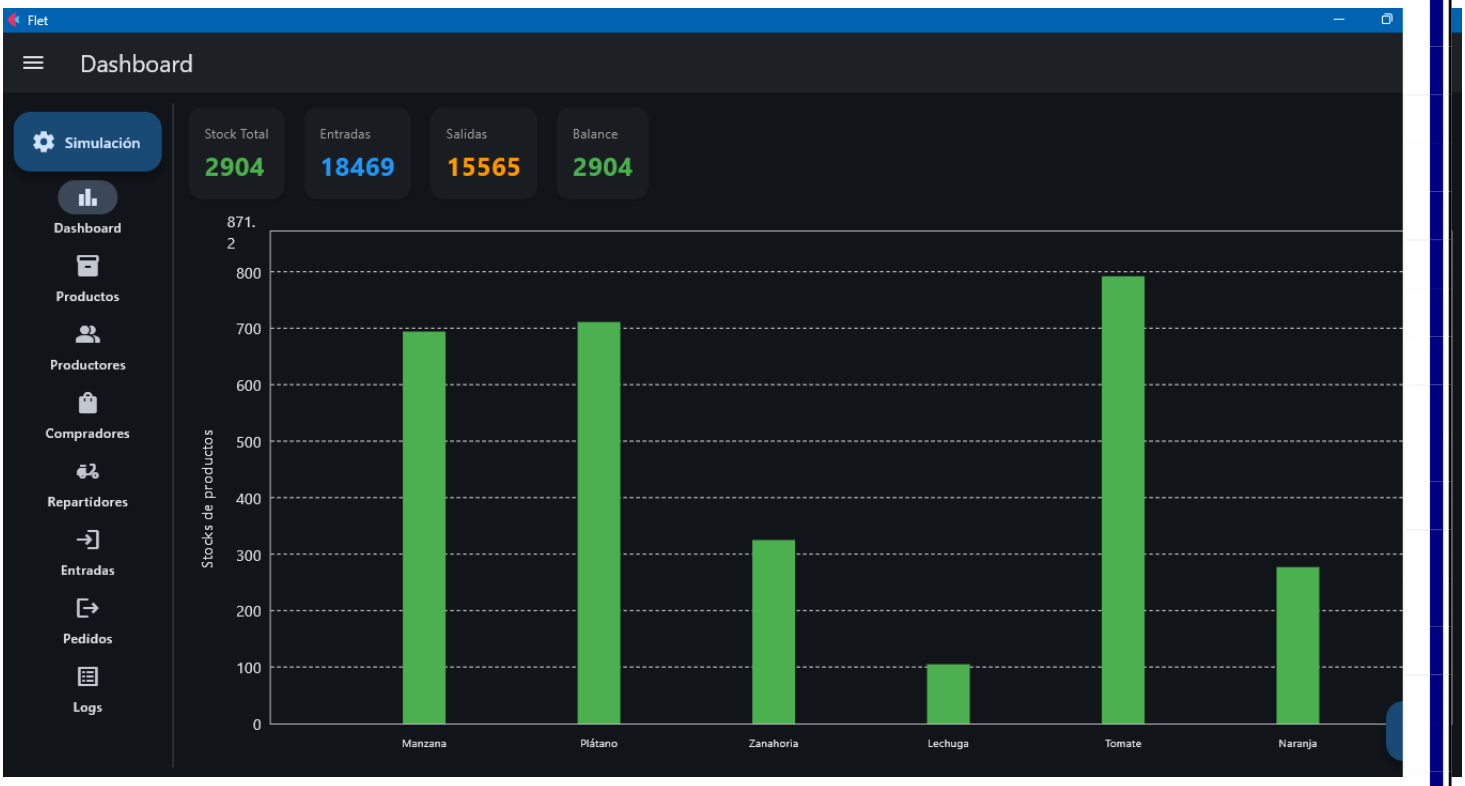
Modelo de datos

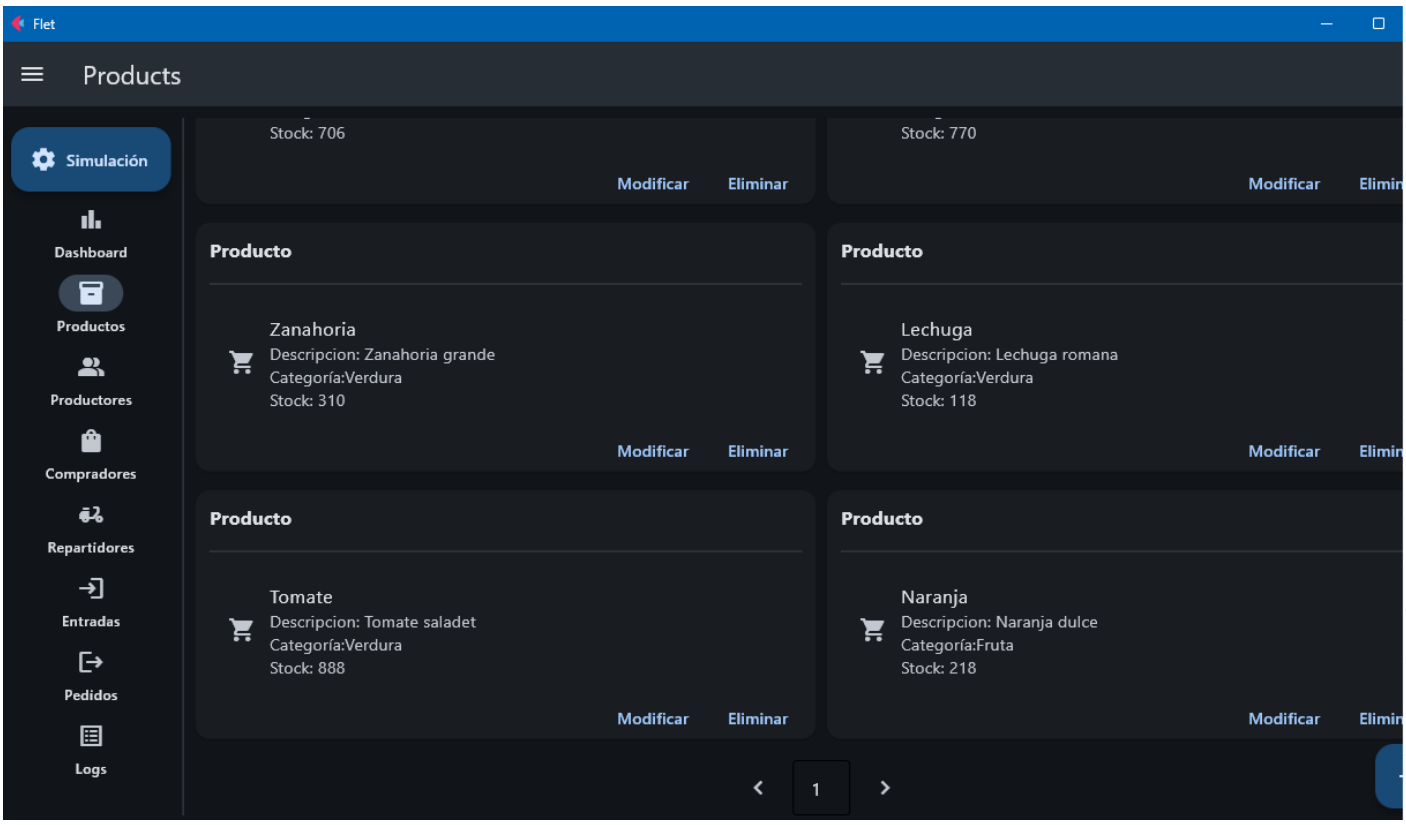
El sistema incluye las siguientes entidades principales:

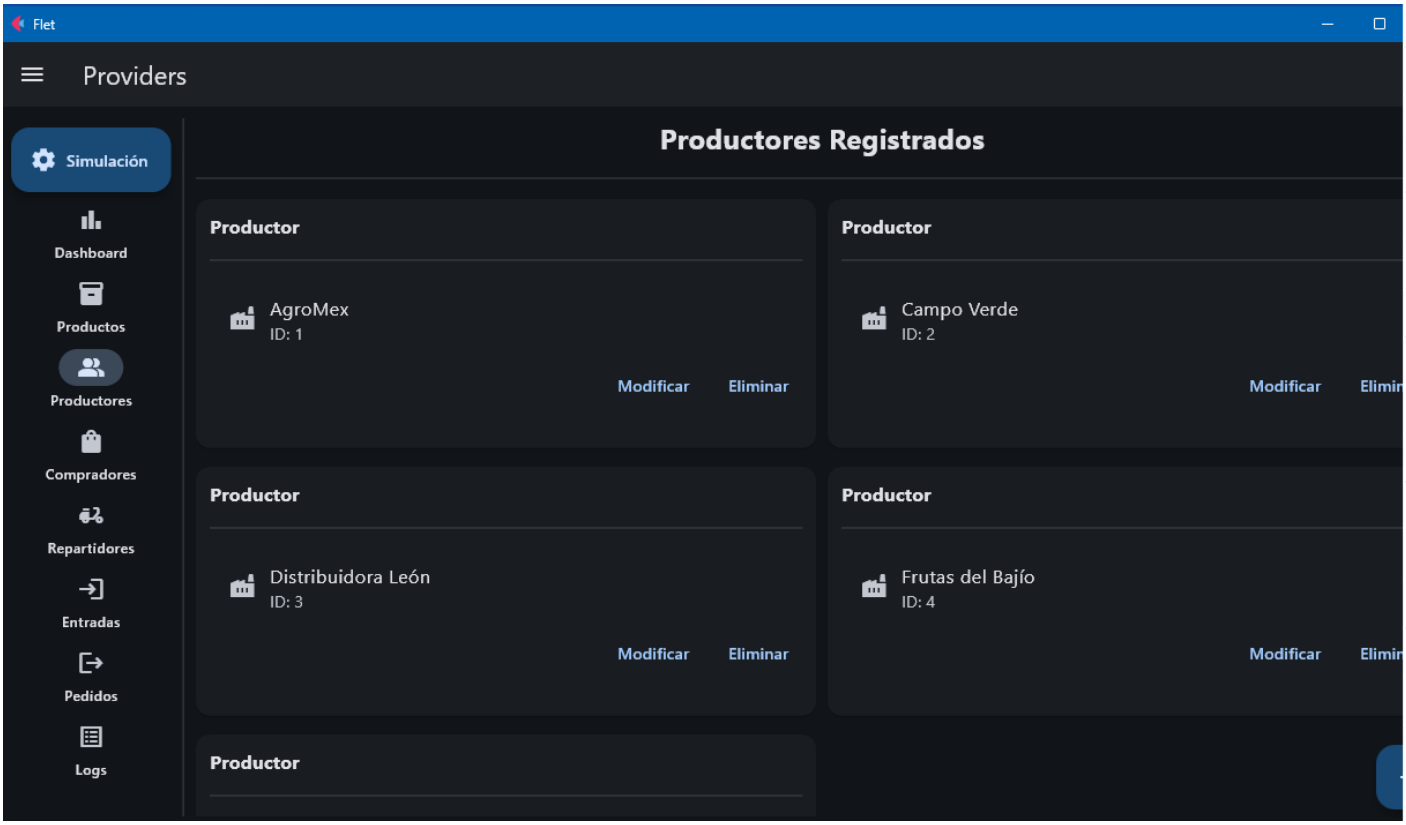
- **Producto**: representa los artículos disponibles.
- **Productor**: proveedores de productos.
- **Entrada**: registros de abastecimiento de productos.
- **Comprador**: clientes que realizan pedidos.

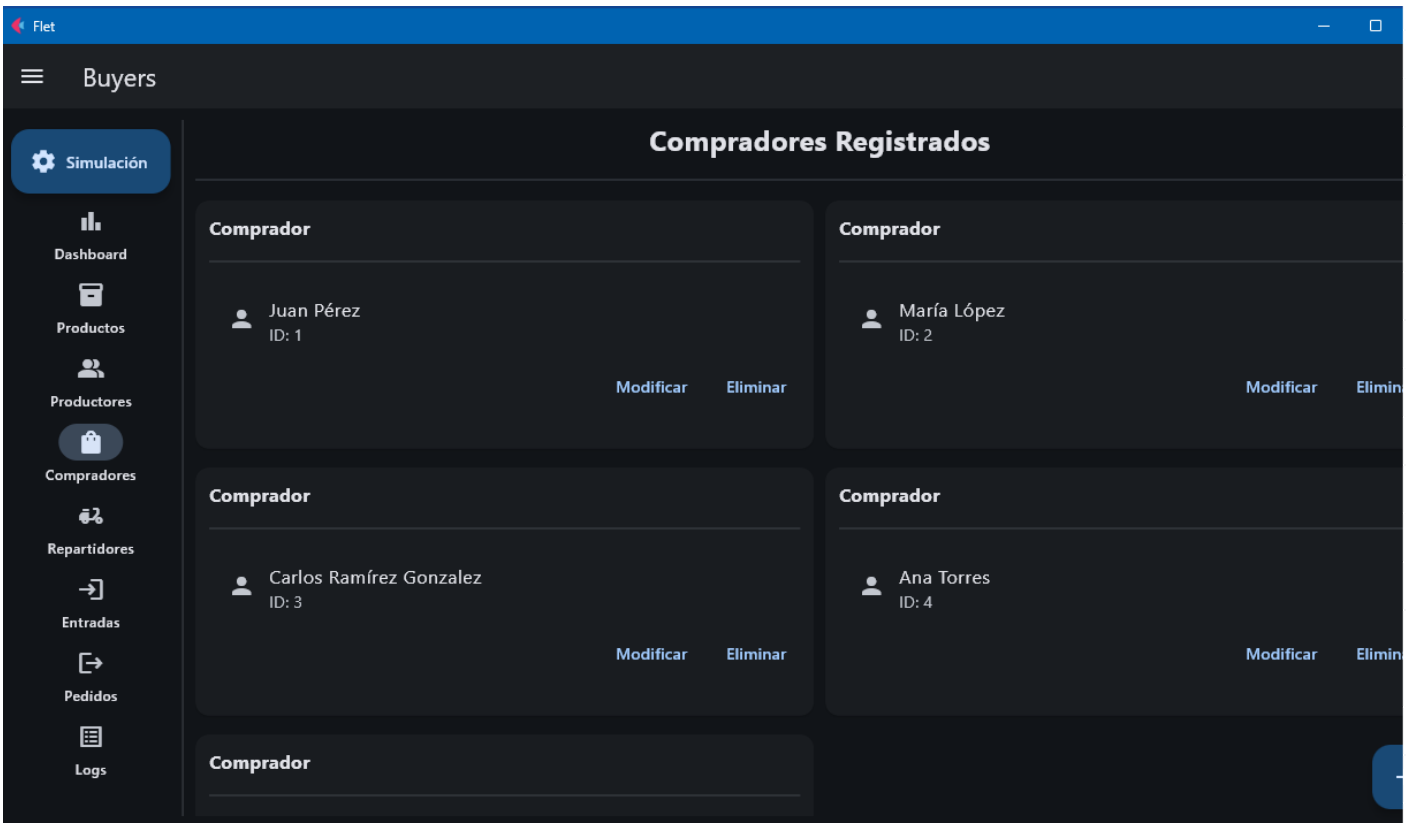
- **Repartidor**: encargados de entregar pedidos.
- **Entrega**: pedidos generados por compradores.
- **DetalleEntrega**: productos incluidos en cada pedido.

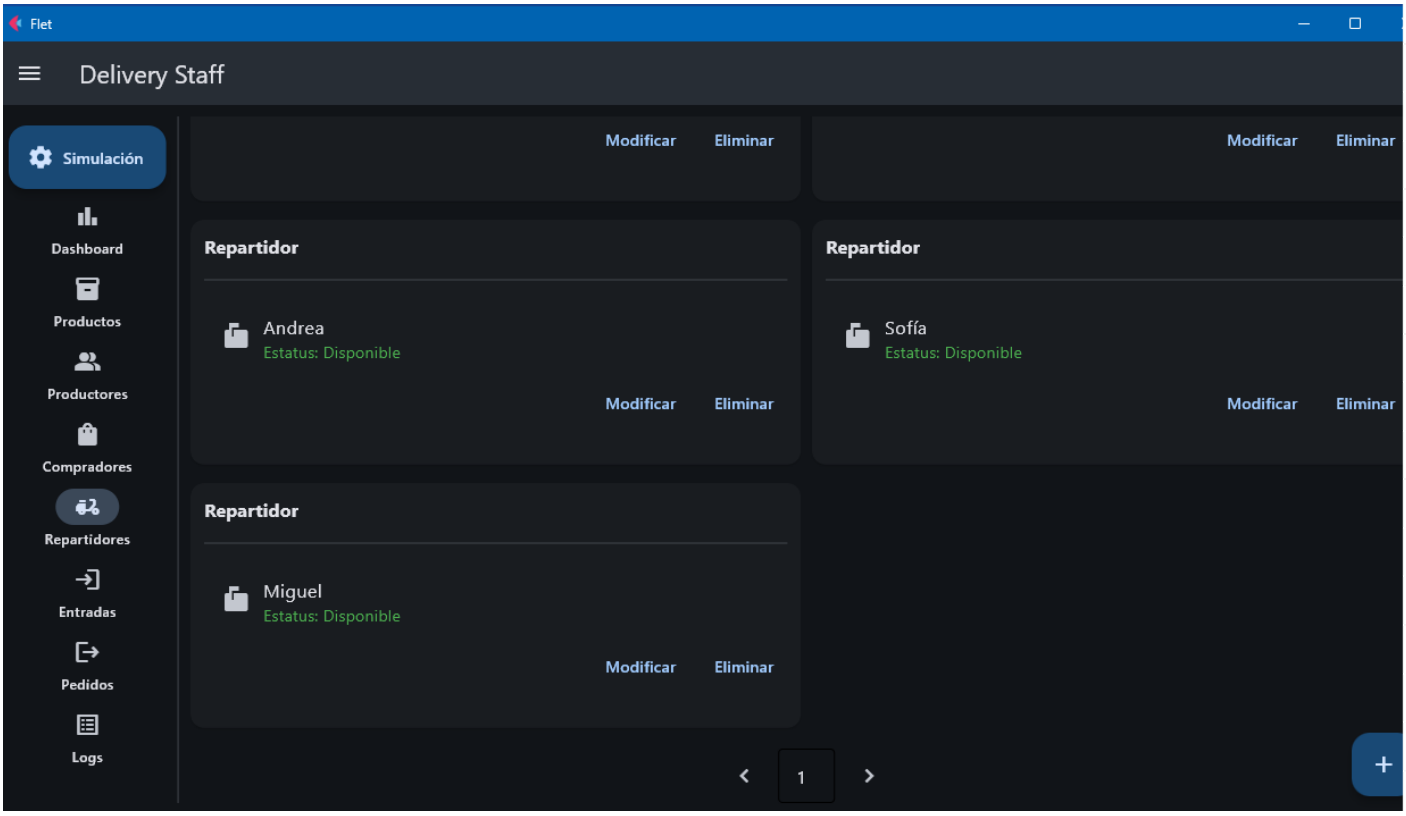
Ejecución:

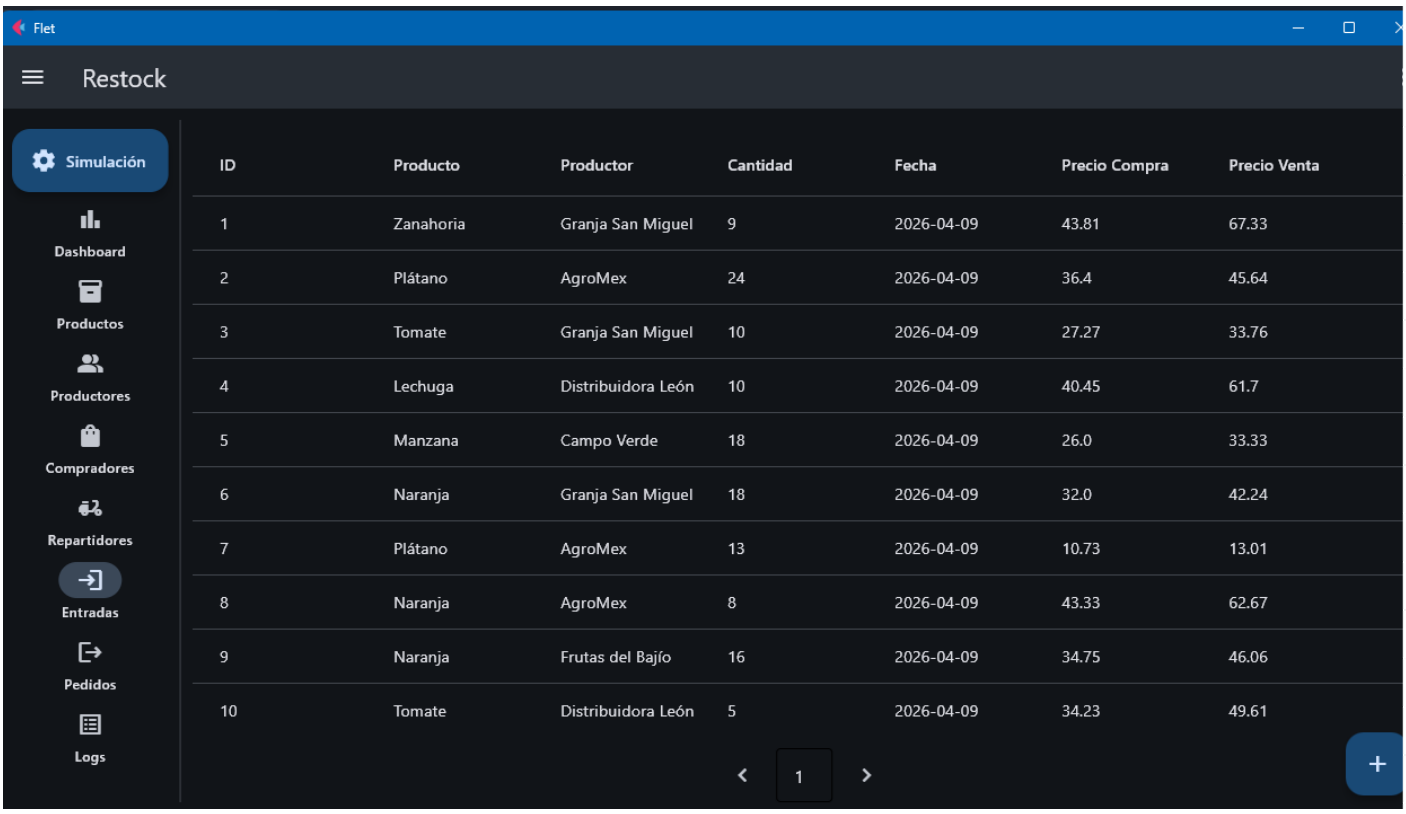



Flet
Products
Simulación
Dashboard
Productos
Productores
Compradores
Repartidores
Entradas
Pedidos
Logs
Stock: 706
Modificar
Eliminar
Stock: 770
Producto
Zanahoria
Descripcion: Zanahoria grande
Categoría:Verdura
Stock: 310
Lechuga
Descripcion: Lechuga romana
Categoría:Verdura
Stock: 118
Tomate
Descripcion: Tomate saladet
Categoría:Verdura
Stock: 888
Naranja
Descripcion: Naranja dulce
Categoría:Fruta
Stock: 218
1

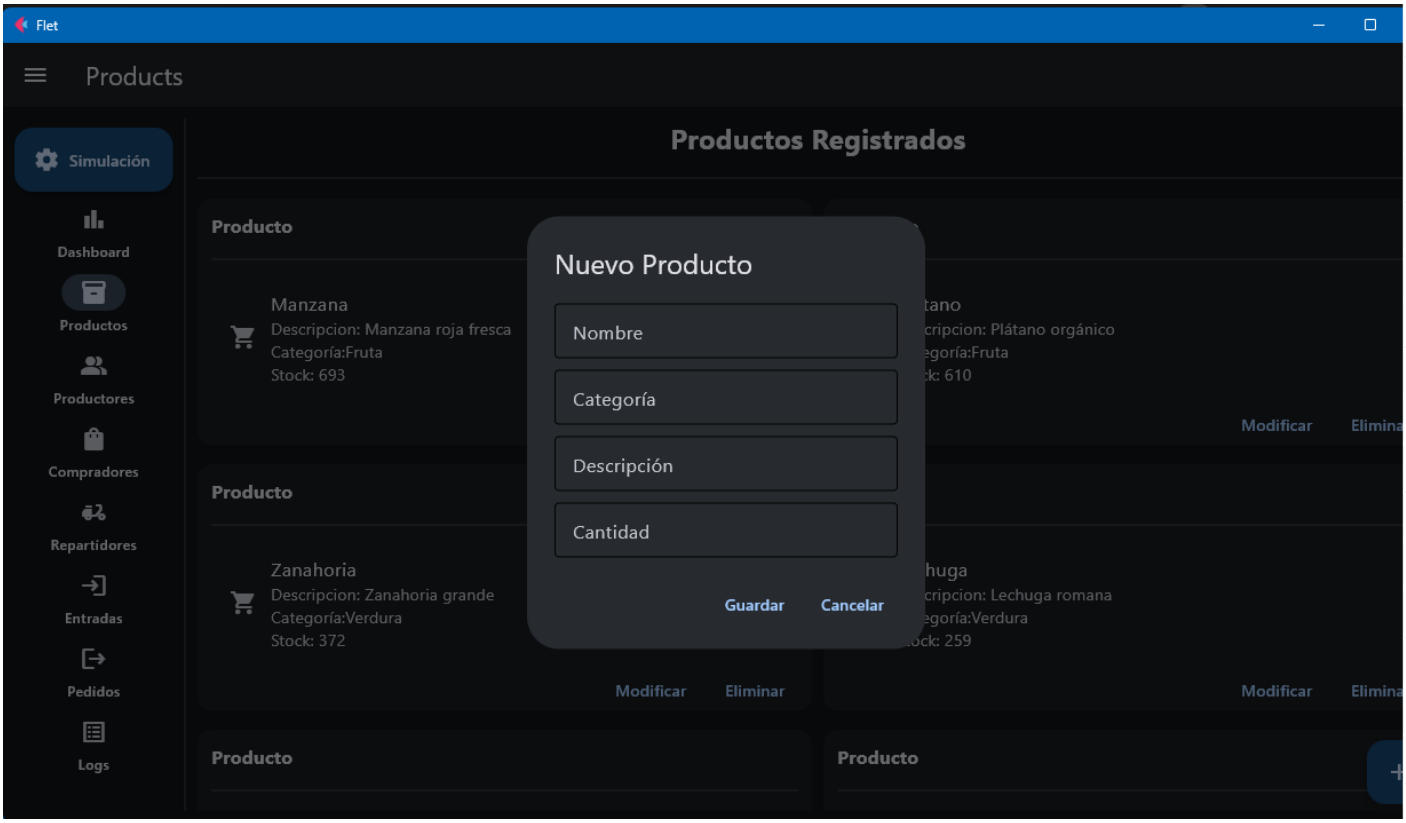

Flet
Providers
Simulación
Dashboard
Productos
Productores
Compradores
Repartidores
Entradas
Pedidos
Logs
Productores Registrados
Productor
AgroMex
ID: 1
Campo Verde
ID: 2
Distribuidora León
ID: 3
Frutas del Bajío
ID: 4
Modificar
Eliminar

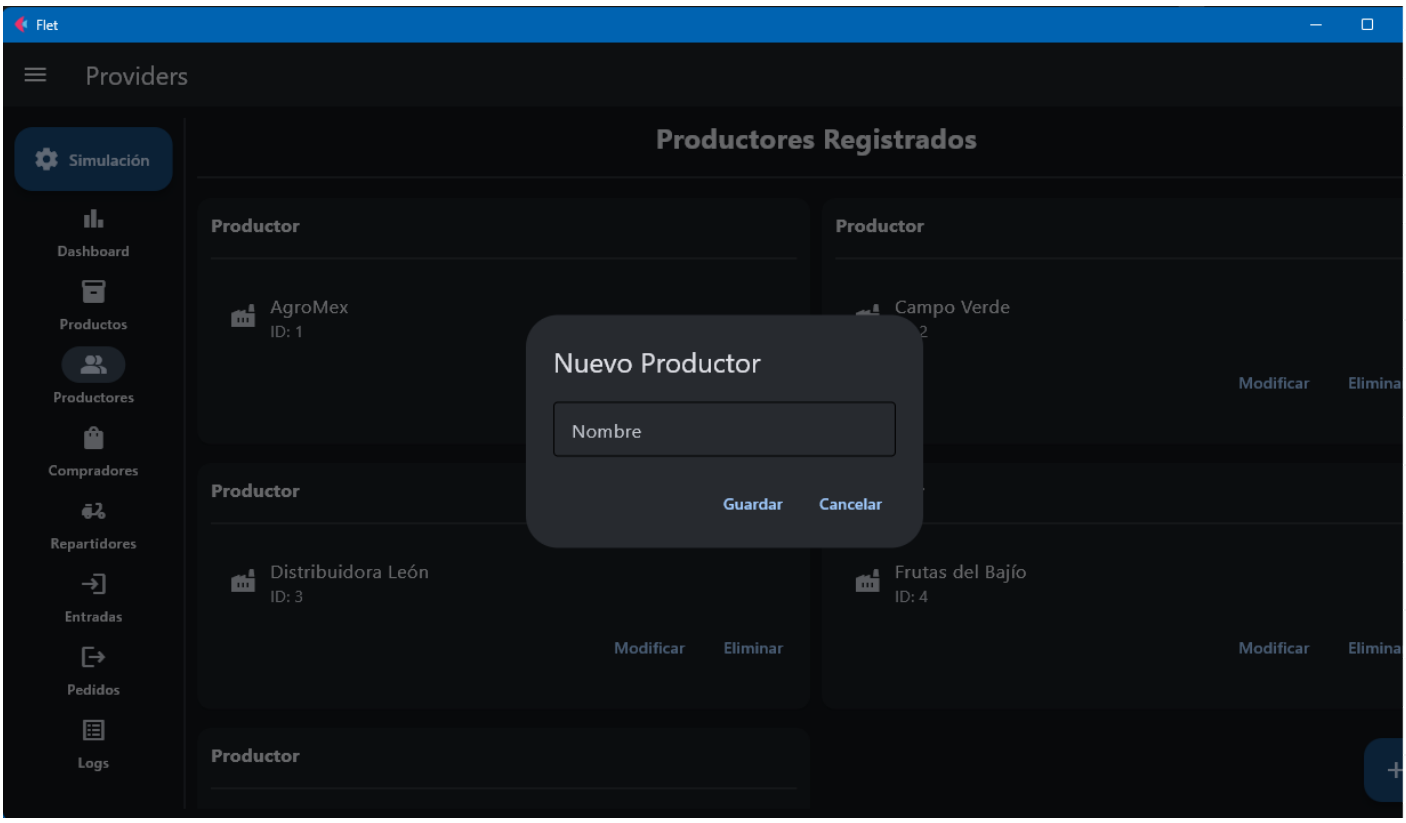

Flet
Buyers
Simulación
Dashboard
Productos
Productores
Compradores
Repartidores
Entradas
Pedidos
Logs
Compradores Registrados
Comprador
Juan Pérez
ID: 1
Modificar
Eliminar
Comprador
María López
ID: 2
Modificar
Comprador
Carlos Ramírez Gonzalez
ID: 3
Modificar
Eliminar
Comprador
Ana Torres
ID: 4
Modificar
Comprador

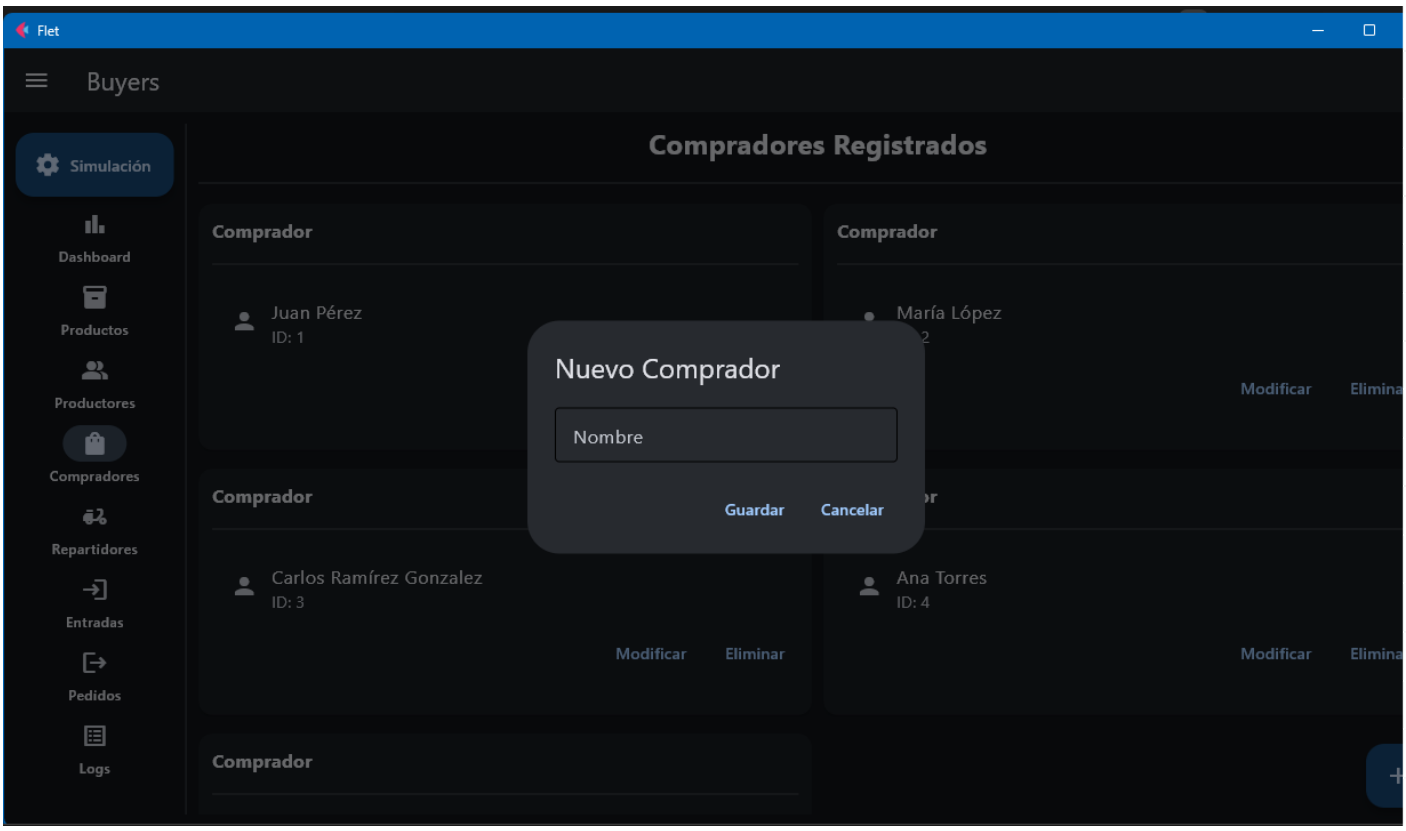

Flet
Delivery Staff
Simulación
Dashboard
Productos
Productores
Compradores
Repartidores
Entradas
Pedidos
Logs
Modificar
Eliminar
Repartidor
Andrea
Estatus: Disponible
Modificar
Eliminar
Repartidor
Sofía
Estatus: Disponible
Modificar
Eliminar
Repartidor
Miguel
Estatus: Disponible
Modificar
Eliminar
1

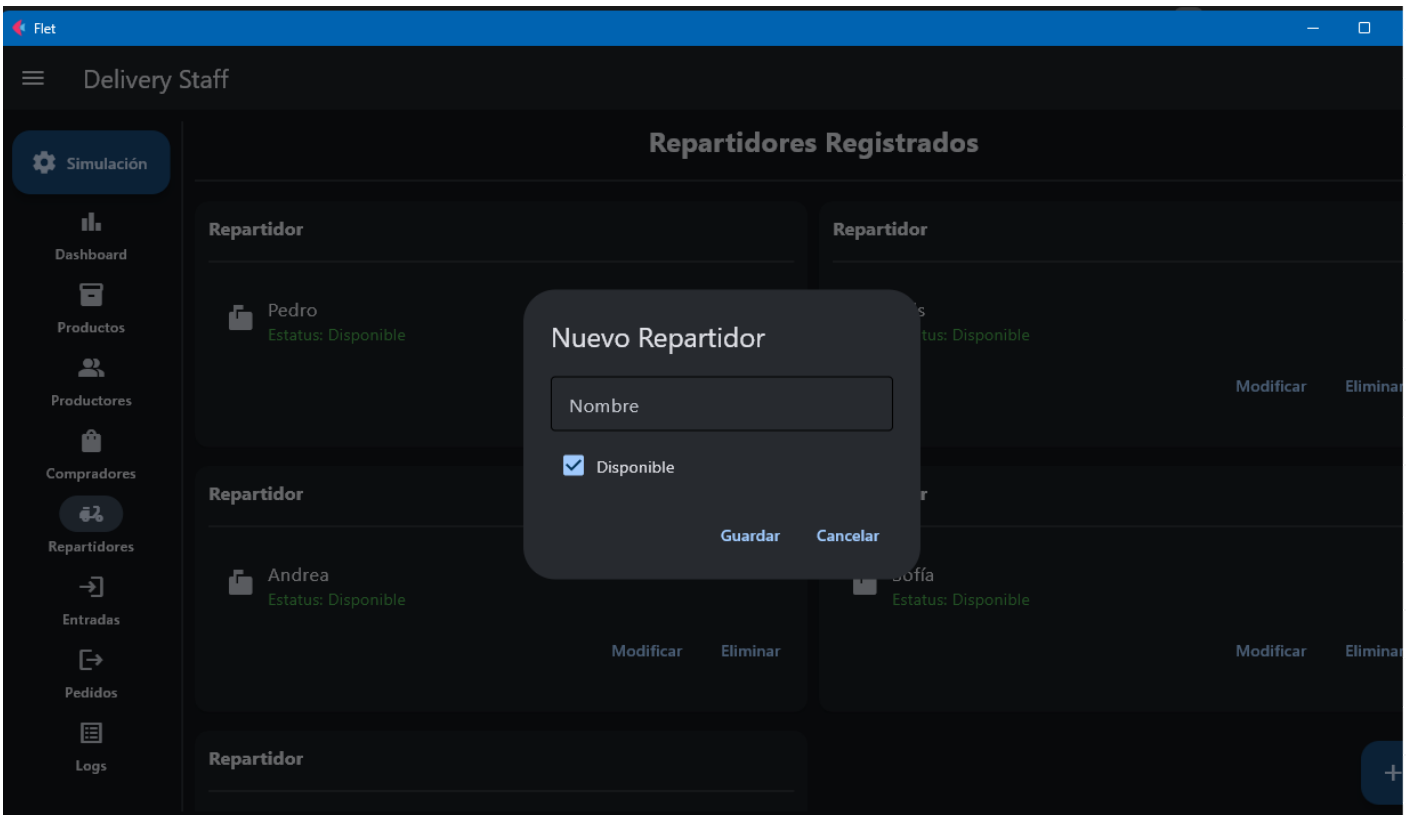


ID	Producto	Productor	Cantidad	Fecha	Precio Compra	Precio Venta
1	Zanahoria	Granja San Miguel	9	2026-04-09	43.81	67.33
2	Plátano	AgroMex	24	2026-04-09	36.4	45.64
3	Tomate	Granja San Miguel	10	2026-04-09	27.27	33.76
4	Lechuga	Distribuidora León	10	2026-04-09	40.45	61.7
5	Manzana	Campo Verde	18	2026-04-09	26.0	33.33
6	Naranja	Granja San Miguel	18	2026-04-09	32.0	42.24
7	Plátano	AgroMex	13	2026-04-09	10.73	13.01
8	Naranja	AgroMex	8	2026-04-09	43.33	62.67
9	Naranja	Frutas del Bajío	16	2026-04-09	34.75	46.06
10	Tomate	Distribuidora León	5	2026-04-09	34.23	49.61

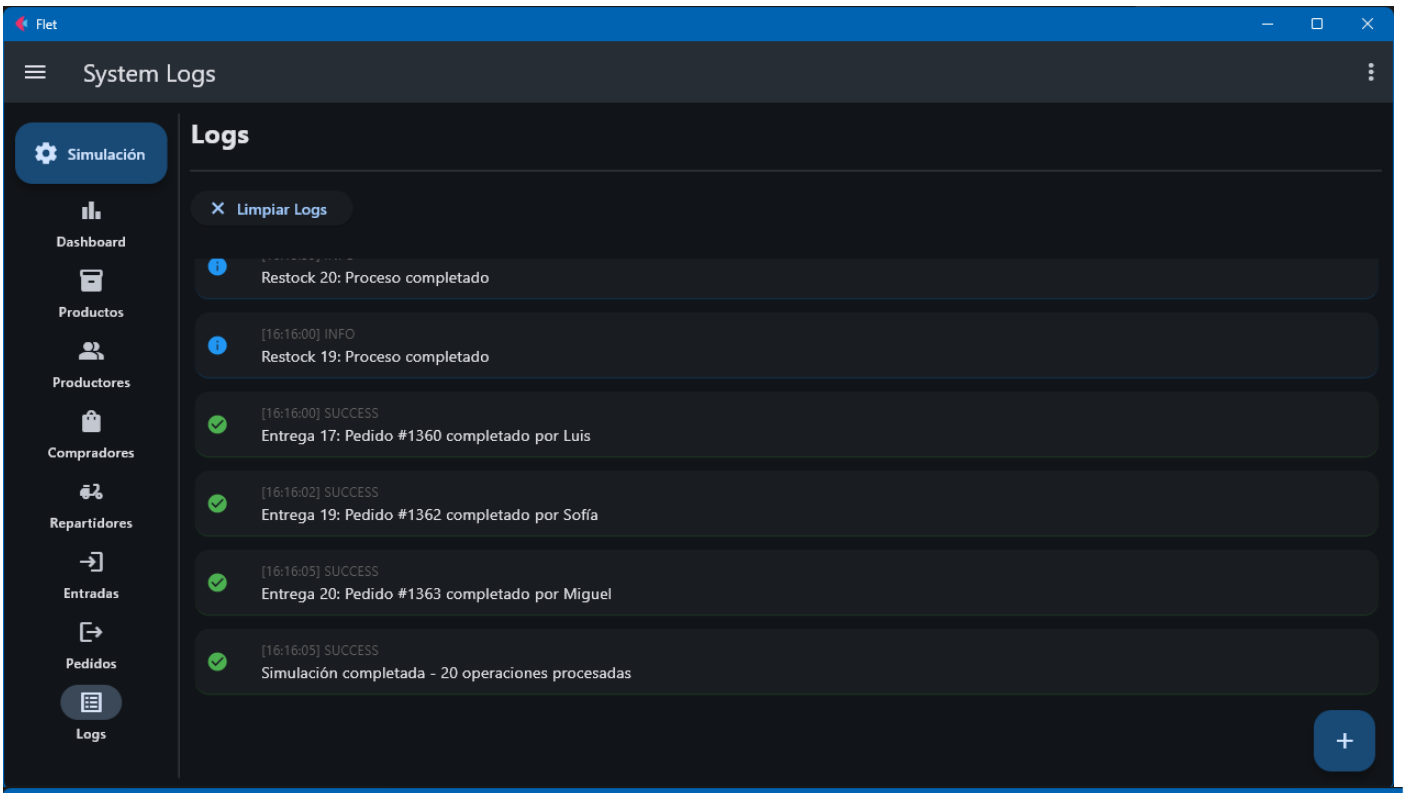


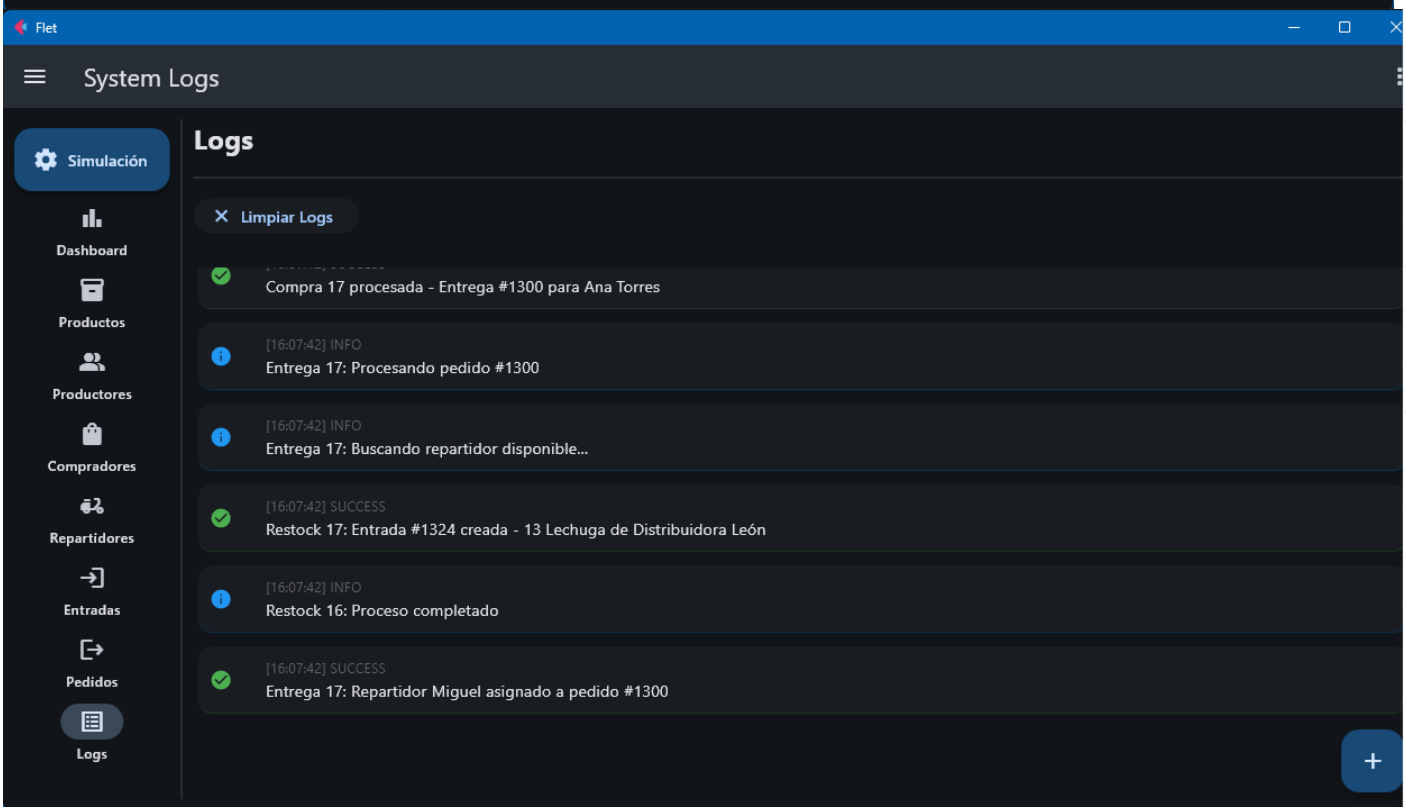

Flet
Providers
Simulación
Dashboard
Productos
Productores
Compradores
Repartidores
Entradas
Pedidos
Logs
Productores Registrados
Productor
AgroMex
ID: 1
Productor
Campo Verde
Modificar
Elimina
Productor
Distribuidora León
ID: 3
Modificar
Eliminar
Frutas del Bajío
ID: 4
Modificar
Elimina
Productor
Nuevo Productor
Nombre
Guardar
Cancelar

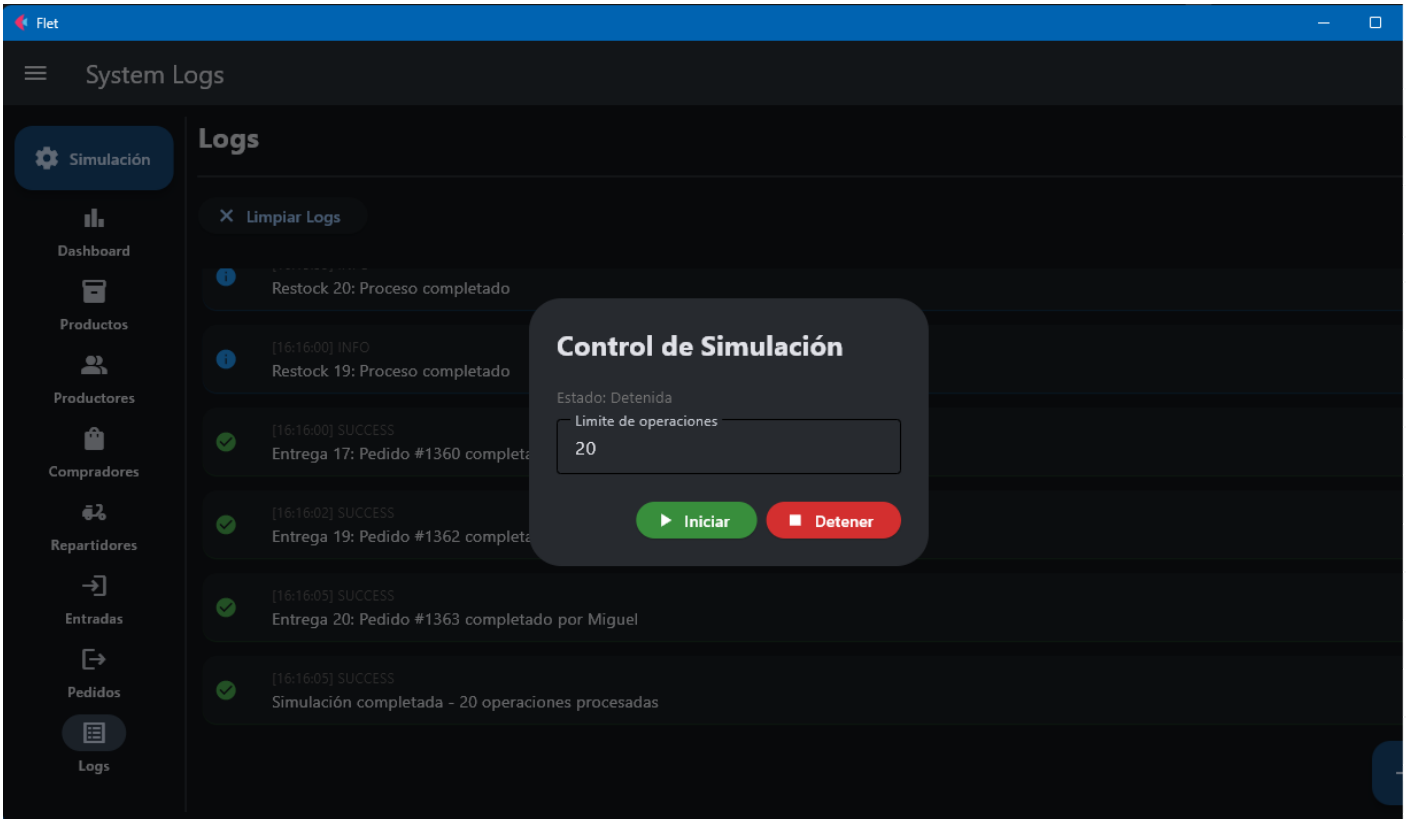

Flet
Buyers
Simulación
Dashboard
Productos
Productores
Compradores
Repartidores
Entradas
Pedidos
Logs
Compradores Registrados
Comprador
Juan Pérez
ID: 1
Comprador
María López
Modificar
Elimina
Comprador
Carlos Ramírez Gonzalez
ID: 3
Modificar
Eliminar
Ana Torres
ID: 4
Modificar
Elimina
Comprador
Nuevo Comprador
Nombre
Guardar
Cancelar



ID	Comprador	Repartidor	Fecha	Entregado	Actions
1	Ana Torres	Andrea	2026-04-09	Sí	
2	Juan Pérez	Andrea	2026-04-09	Sí	
3	María López	Sofía	2026-04-09	Sí	
4	Luis Fernández	Miguel	2026-04-09	Sí	
5	María López	Pedro	2026-04-09	Sí	
6	Ana Torres	Miguel	2026-04-09	Sí	
7	Luis Fernández	Andrea	2026-04-09	Sí	
8	Ana Torres	Miguel	2026-04-09	Sí	
9	Luis Fernández	Miguel	2026-04-09	Sí	
10	Ana Torres	Pedro	2026-04-09	Sí	


Flet
System Logs
Simulación
Dashboard
Productos
Productores
Compradores
Repartidores
Entradas
Pedidos
Logs
Logs
Limpiar Logs
Restock 20: Proceso completado
[16:16:00] INFO
Restock 19: Proceso completado
[16:16:00] SUCCESS
Entrega 17: Pedido #1360 completado por Luis
[16:16:02] SUCCESS
Entrega 19: Pedido #1362 completado por Sofía
[16:16:05] SUCCESS
Entrega 20: Pedido #1363 completado por Miguel
[16:16:05] SUCCESS
Simulación completada - 20 operaciones procesadas


Flet
System Logs
Simulación
Dashboard
Productos
Productores
Compradores
Repartidores
Entradas
Pedidos
Logs
Logs
Limpiar Logs
Compra 17 procesada - Entrega #1300 para Ana Torres
[16:07:42] INFO
Entrega 17: Procesando pedido #1300
[16:07:42] INFO
Entrega 17: Buscando repartidor disponible...
[16:07:42] SUCCESS
Restock 17: Entrada #1324 creada - 13 Lechuga de Distribuidora León
[16:07:42] INFO
Restock 16: Proceso completado
[16:07:42] SUCCESS
Entrega 17: Repartidor Miguel asignado a pedido #1300


Flet
System Logs
Simulación
Dashboard
Productos
Productores
Compradores
Repartidores
Entradas
Pedidos
Logs
Logs
Limpiar Logs
Restock 20: Proceso completado
[16:16:00] INFO
Restock 19: Proceso completado
[16:16:00] SUCCESS
Entrega 17: Pedido #1360 completa
[16:16:02] SUCCESS
Entrega 19: Pedido #1362 completa
[16:16:05] SUCCESS
Entrega 20: Pedido #1363 completado por Miguel
[16:16:05] SUCCESS
Simulación completada - 20 operaciones procesadas
Control de Simulación
Estado: Detenida
Limite de operaciones
20
Iniciar
Detener

Sistema de Control de Productos
Dashboard
Alerta stock bajo: Frutitlla - Stock: 2
Pera - Stock: 1
Refrescar
Simulación
Dashboard
Productos
Productores
Compradores
Repartidores
Entradas
Stock Total
3854
Entradas
26704
Salidas
22866
Balance
3838
Stocks de productos
1.2K
1.1K
1K
900
800
700
600
500
400
300
200
100
0
Manzana
Plátano
Zanahoria
Lechuga
Tomate
Naranja
Tomate ver
Frutitlla
Pera

Prompts:

Para agilizar el proceso de creación, planeación, documentación y desarrollo del sistema se hizo uso de algunos modelos de inteligencia artificial desde Chatgpt 5 de openAI para realizar un pequeño esbozo y planeación de ideas y conceptos del sistema, github copilot de Microsoft con un modelo de Claude Haiku 4.5 y Gemini 3.1 low de google para documentación, así como resolución de errores e integración de algunas partes de código:

Prompt 1

Basandote en los siguientes requerimientos:

Desarrollar una aplicación que simule un sistema concurrente del mundo real utilizando programación con hilos en Python. El tipo de sistema es libre, puedes elegir el escenario que desees implementar, debe incluir: ü Modelo Productor–Consumidor ü Interfaz gráfica con Tkinter (Visualización gráfica) ü Base de datos SQLite (Registro de eventos) ü Lock para sincronización ü Recursos compartidos ü Hilos concurrentes ü Medición de rendimiento ü Cola Queue ü Funcionamiento general

quiero que me proporciones ideas para realizar este proyecto utilizando flet como framework de UI

Prompt 2

Últimamente he estado trabajando con SQLAlchemy y Flask para el manejo de bases de datos con ORMs que son objetos que reflejan la estructura del modelo y tabla en la base de datos hay alguna manera de implementar los ORMS en la arquitectura actual de mi sistema

Prompt 3

Ya estoy comenzando a realizar pruebas de la implementación y creación de la base de datos con las migraciones y alembic, pero tengo el siguiente error podrías indicarme a que se refiere? The application encountered an error: Parent instance <Entrega at 0x19b17a7b6a0> is not bound to a Session; lazy load operation of attribute 'detalles' cannot proceed (Background on this error at: https://sqlalche.me/e/20/bhk3)

Prompt 4

Oye necesito realizar unos cambios en algunas columnas para la base de datos, pero quiero practicar el concepto de migraciones, para no tener que recrear la base dedatos cada vez que tenga datos, de que manera se generan las migraciones con alembic y SQLAlchemy

Prompt 5

Excelente, ya logré implementar los modelos basándome en la estructura que diseñe, ahora con respecto al modelo o a las funciones en si, encuentras algún error de sintaxis o de logica que sea necesario corrergir?

Prompt 6

estoy pensando en implementar una pequeña función que divida los resultados de la consulta para implementar algún tipo de paginación.

def pagination(elements:List=[i for i in range(120)],elements_per_page=20): pagination=[] prev=0 for next in range(elements_per_page,len(elements)+1,elements_per_page): pagination.append(elements[prev:next]) prev=next return pagination

de momento esto es lo que tengo, este es un concepto inicial para poder segmentar la información, así que mi idea es llamar allí a cualquiera de las funciones que devuelva una lista de elementos y luego devolver una lista segmentada. que te parece?

Prompt 7

Ok, mira te adjunto la estructura principal del código que tengo actualmente, puedes analizarlo y darme algunas sugerencias, retroalimentación para mejorar el funcionamiento, errores queencuentres o cualquier inconsistencia que puedas identificar?

Prompt 8

basándote en las funciones y el modelo creado de la base de datos genera una función que sirva para poblar la base de datos específicamente las tablas de productores, productos, repartidores y compradores, con registros iniciales listos para crear una simulación, utiliza las funciones definidas para el modelo

Prompt 9

Estaba pensando en agregar un método para acceder a los atributos definidos para una clase, de la siguiente manera:

```
def attributes(self):

        return ["id_productor","nombre","activo"]
```

para poder facilitar la validación de atributos que se reciben para la verificación y validación de datos, crees que haya una manera más directa de impleméntalo a todas las clases de mis modelos de la base de datos

Prompt 10

Me ayudas a resolver este bug de la paginación, cuando dejo el textfield que contiene el número de página en blanco y presiono alguna de las flechas para avanzar o retroceder obtengo el siguiente error

Unhandled error in 'on_click' handler Traceback (most recent call last): File "D:\Python Code\Control de procesos\proyecto_final\venv\Lib\site-packages\flet\messaging\session.py", line 335, in

dispatch_event await control._trigger_event(event_name, event_data) File "D:\Python Code\Control de procesos\proyecto_final\venv\Lib\site-packages\flet\controls\base_control.py", line 478, in _trigger_event event_handler(e) ~~~~~~~~~~~~~~^^^ File "d:\Python Code\Control de procesos\proyecto_final\views\pagination.py", line 15, in next_pressed if int(current_page.value)+1>pagination["total_pages"]: ~~~^^^^^^^^^^^^^^^^^^^^ ValueError: invalid literal for int() with base 10: '' when you leave the textfield blank and press one the arrows this happens

Prompt 11

He finalizado el diseño de la arquitectura de la base de datos y los modelos, basándote en la idea proporcionada para el sistema podrías sugerirme ideas de implementación para el layout de la barra lateral y menú de opciones a implementar del sistema

Prompt 12

Quiero implementar una sección de dashboard con algunas estadísticas del stock y los productos, ya hace algun tiempo que cree graficas de datos en flet en versiones como la 0.28.1, pero por lo que he estado revisando el manejo de estas ha cambiado, podrías proporcionarme un ejemplo funcional dinamico de una grafica de datos basándote en la documentación oficial de flet https://flet.dev/docs/controls/charts/barchart?

Prompt 13

Revisando nuevamente la documentación de flet para la versión 0.86 he notado que además de existir el componente DataTable de versiones anteriores hay un nuevo componente llamado DataTable2 el cual es muy similar pero cuenta con filtros que permiten ordenar las filas de las tablas, podrías ayudarme a implementar este componente con filtros?

Prompt 14

Hola necesito ayuda con este error dentro de mi código, no entiendo por que ocurre, me lo puede explicar y me puedes ayudar a resolverlo?

Traceback (most recent call last): File "d:\Python Code\Control de procesos\proyecto_final\venv\Lib\site-packages\flet\app.py", line 354, in on_session_created main(session.page) ~~~~^^^^^^^^^^^^^^ File "d:\Python Code\Control de procesos\proyecto_final\views\simulation.py", line 91, in simulation_view bgcolor=ft.Colors.SURFACE_VARIANT ^^^^^^^^^^^^^^^^^^^^^^^^^ File "d:\Python Code\Control de procesos\proyecto_final\venv\Lib\site-packages\flet\utils\deprecated_enum.py", line 49, in __getattr__ return super().__getattr__(name) ^^^^^^^^^^^^^^^^^^^ AttributeError: 'super' object has no attribute '__getattr__'. Did you mean: '__setattr__'?

Prompt 15

Ok hasta ahora mi sistema cuenta con concurrencia con hilos, ya tengo la simulación creada interfaz gráfica con un CRUD básico creado, solo me hace falta implementarla, qué opinas de implementación actual y los cambios que he hecho hasta ahora, que recomendaciones me harías?

Prompt 16

Estoy trabajando en la vista del dashboard me ayudarías a mejorarla? Principalmente se muestra un grafico del stock de productos, me gustaría que cuando el stock de algún producto sea bajo sea notificado en la interfaz del dashboard me puedes ayudar a implementar este feature?

Prompt 17

Me podrías ayudar a implementar un crud de manera sencilla para cada una de las entidades que interactúan en el sistema, como productos, repartidores, proveedores, clientes, etc. Puede de preferencia basarte en las funciones que generé dentro del archivo de models.py?

Prompt 18

Estoy teniendo problemas para manejar la simulación de entrada y salida de productos, inicialmente estaba trabajándolo sobre el mismo hilo pero me dí cuenta de que al hacer esto la interfaz se detenía, por lo que decidí implementar un hilo demonio para que no intervenga con el flujo de la interfaz, pero la vista donde se muestran los logs no se actualiza adecuadamente, además la funcionalidad para detener la simulación no esta implementada adecuadamente me ayudas a revisarla?

Prompt 19

Encontré un bug mientras intentaba hacer algunas pruebas con la interfaz grafica al correr ua simulación y revisar el dashborad me encontré con la siguiente excepción RuntimeError: dictionary changed size during iteration, puedes decir que podría significar este error basándote en el código proprocionado y el contexto anterior?

Prompt 20

Excelente ya he hecho las pruebas suficientes y ya no encontré ningún bug durante la ejecución del programa basado en todo el contexto anterior podrías ayudarme a construir el archivo readme con la documentación necesaria del sistema, para su ejecución y descripción de este

Código

Repositorio del proyecto: simulacion-sistema-concurrente. Este espacio contiene el código fuente, las vistas y los requerimientos necesarios para la ejecución del sistema. Además, se incluyen diagramas y documentación gráfica para facilitar la comprensión de la arquitectura.

Link al repositorio: https://github.com/DaveChessington/simulacion-sistema-concurrente